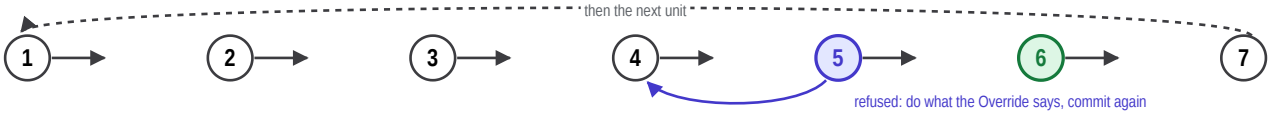
Charter at the desk

Package 0.4.4. Front: the working day, a new project, a migration. Back: the reference.
`charter` = `python3 .charter/bin/charter` (project root), `charter-pkg` = `python3 ~/Projects/charter/bin/charter-pkg`

O the owner acts · A the agent acts · *refused* hard gate, names its override · *warns* logged, lets it pass · *passes* · *" phrase "* what you say to the agent

Every day: the unit loop

One unit per deliverable. The agent runs steps 1 to 7; the teal lane is yours. Source: `docs/daily-use.md` § The loop.



Open a session

Read the banner:

```
charter 0.4.4, track prototype, opt-ins: none
```

- `charter next`: active unit, road, next units, what waits on you. Writes nothing.
- Hook not active: `charter setup` once.
- On 0.4.2 or older you see no banner: `charter config`.

Pick or write the unit

"start unit <id>" or *"write a unit for …"* (skill `charter-unit`).

Ready when each **AC** names a probe that **fails now** (`(guard)` if it protects what works), `perimeter:` lists the paths, `scope:` is set on client, `rail:` where there are rails.

Start it

```
charter unit start <id>
charter probe <id>
```

```
→ exit 1 (ModuleNotFoundError …)
```

That failure is the **baseline**, the session's first `## Log` line. A probe passing now proves nothing.

Work

- Outside `perimeter:`? Add `- path: why` under `## Outside perimeter`.
- *"record the decision"*: `charter/decisions/<slug>.md`, cited in code as `charter:<slug>`.
- Frozen file (800 lines; 400 for governance): move code out.
- Unsold work: a change request.
- Dated `## Log` line when something lands.

Commit

On a branch. The hook runs `charter check --staged`:

```
charter check: 0 blocking, 0 warning(s) [staged, …]
```

Refused: each line ends with **Override:**; do that. **Never** `--no-verify` or a Bash workaround: CI runs the same checks.

Close it

"finish the unit"

1. `charter probe <id>` → exit 0
2. tick `- [x] AC-n`
3. `## Evidence`: SHA, `path:line`, the `$ command` lines
4. `status: done`, closing log line
5. `charter unit stop`

Final message cites the same, or the Stop hook blocks it once.

Report

"where does this stand?" (skill `charter-progress`)

`charter ledger fold` `charter synth --write`
Writes `charter/progress.md`, or refuses if a line is unanchored. Never hand-edited. A client gets scope and accepted units.

O Only you

Commit what was staged for you (install, upgrade, config, rails): `git commit` in your own terminal, or `CHARTER_OWNER=1 git commit` after `!` in Claude Code.

Re-plan: add or reorder rows in `charter/rails.md`.

Approve a change request: `status: approved`, `approved_by:`, `approved_on:`. **Retire a decision**: `status: superseded`, `superseded_by:`.

Push, open, merge with a merge commit, which keeps evidence SHAs. **Deploy.**

Let a frozen file grow: a `kind: split` unit, or a `size_exceptions` entry (budget, `because:`, `expires:`).

Decide proposed decisions; answer the agent's commitments (price, dates, data, releases).

Does the project already have code, taskets or documents?

no: install directly · yes: survey first

Install steps 1 and 2, either way: client track only under a signed devis or contract, else prototype; `--track` is required.

A new project

Say *"install charter on <project>"*. The agent follows `docs/install.md` and stops with the files staged. `$P` is the project; counts are from the 0.4.4 rehearsal.

0. **Preconditions.** A git repo with a commit; a clean tree; no other session in it; `charter-pkg status $P` → `installed: none`.
3. **Branch.** `git -C $P switch -c charter/install`, never the main line.
4. **Dry run.** `charter-pkg install $P --track prototype --dry-run` → `dry run: 35 change(s)`. Watch for `KEEP git hooks` (step 6) and `IGNORED by` (ask the owner).
5. **Install.** The same without `--dry-run`. Run again: `already installed … nothing to do`.
6. **Only after KEEP git hooks:** add `python3 -B .charter/bin/charter check --staged` to the lefthook, pre-commit or husky hook.
7. **Stage by name** (34 files), never `git add -A`.
8. **You commit.** Set `"owner"` in `charter/config.json` to your git author name; `charter config` shows no `problem:`; `git commit -m "charter 0.4.4"`. The agent's own attempt gets 16 blocking: expected.
9. **Verify.** `charter check --all` → 0 blocking; `charter-pkg status $P` → 0 modified, missing, differing; a new session shows the banner; a fresh clone passes `check --all --strict`.

O You push and merge (merge commit). Each other clone: `charter setup` once. Client track: you write `charter/scope.md` from the signed devis, one item per sold deliverable, no prices.

Where the gates run, from then on

Claude Code hooks — each Edit, Write, NotebookEdit refuses early, with the fix. not Bash, not Codex

→ **git pre-commit** — every commit, any agent judges the index. --no-verify skips it

→ **CI** — at merge, clean clone checks who authored. owner-only changes

Sources: `docs/install.md` steps 0-9, § Every other clone; `docs/migration.md` Paths A and B; `docs/gates.md` § Git hook, § CI.

Migrating an existing project

Say *"put charter on <project>"*: `CharterAdmin` runs its `Migrate` workflow. Nothing touches the project before your go-ahead.

1. **Survey, read-only.** `charter-pkg survey $P --track prototype --out <dir outside>` writes `survey.md` and `plan.md` and rehearses the install in a throwaway clone. `--test-cmd "<tests>"` exit 0: probes can run at commit.
2. **Settle what the facts settle.** A lookup is not a decision; a proposal is not a scope.
3. **Your calls on a Slate**, including the go-ahead for the real tree.
4. **Rehearse the whole plan** in a kept clone (`--keep-clone`), the only place `CHARTER_OWNER=1` is allowed.
5. **Execute, after the go-ahead,** in a worktree on `charter/install`: one step, one commit, one check; a red check stops it. Owner-only steps stop staged for O.
6. **You push and merge.** The agent reports each step with its check, and what is left.

What happens to the old work

- Taskets with neither `## Acceptance` nor `scope:` are listed in `charter/legacy-taskets.txt`; closing one only warns.
- Units done before the install show under **Done before charter** in `charter synth`, not as failures.
- Triage by batch: each open tasket gets `scope:`, `perimeter:` and probes, or is parked with a reason; its id leaves the legacy list.
- `CLAUDE.md` rulings become decisions; progress files are archived, since `charter synth` replaces them; BMAD plans stay as inputs, stories stop.

Sources: `personal/claude-home/Skills/CharterAdmin/workflows/Migrate.md`; `docs/migration.md` Paths C-E; `is_charter_unit` in `package/.charter/lib.py`.

Once a week

`docs/daily-use.md` § The owner's weekly look

- `charter next`: **Waiting on the owner** lists proposals, finished rails, files staged for you.
- `charter synth`, no `--write`: adds gate activity.
- `charter log`: 30 days of refusals and warnings. A warning that keeps catching real problems is evidence for a gate; one that fires on nothing is noise.
- No new gates before two or three projects finish a full cycle (your ruling, 2026-09-24).
- `charter config`, then opt in only when the project can bear it: `probes.at_commit/in_ci` when probes run offline and fast; `probes.base_must_fail` when they reliably fail first; `review.required` with an independent reviewer; `rails.required` when every open unit names a rail.

Charter reference

What a unit looks like, what counts as evidence, what to do when a gate refuses, and every command.

(O) owner (A) agent refused warns writes nothing

A unit, `.taskets/<id>.md`

```
---
id: 20260925-greet
title: A greeting function
status: next (1)
scope: S1 (2)
rail: r1-greet
perimeter: [app/**] (3)
kind: work
---
## Acceptance (4)
- [ ] AC-1: greet() returns hello
  — probe: `python3 -c "…"`
- [ ] AC-2: (guard) old API intact
  — probe: `pytest tests/api -q`
## Evidence (5)
- a76f695 adds greet (app/greet.py:1-2)
- `$ python3 -c "…"` → exit 0
## Log (6)
- 2026-09-25 09:00 started; baseline
  AC-1 → exit 1
## Outside perimeter
```

1. `now next later blocked done`; closed without evidence, honestly: `parked`, `skipped`. Comments go on their own line, never after a value.
2. `scope:` an in-scope item or an approved change (client track). `rail:` an open item of `charter/rails.md`.
3. Paths it may touch. Outside them, a note under `## Outside perimeter`: refused at commit on client, warns on prototype.
4. One binary probe per line, written before the code, failing before the work. Once the unit leaves `next`/`later`, changing a line needs a dated log line.
5. A commit, a `path:line`, the `$ command` lines `charter probe` printed. With `review.required`: `- review: <who, model> — <verdict> — <anchor>`.
6. Append-only and dated. The next session resumes from here, not from the transcript.

Sources: `package/charter/templates/unit.md`; `package/.claude/rules/charter.md:7-25`; `TERMINAL_STATUSES` in `package/.charter/lib.py`.

Evidence the done gate accepts

- counts a commit SHA that resolves
- counts `$ command` → exit 0, as run
- counts `path:line` of the change, beside a commit or run
- counts every `- [x]` ticked after its probe passed
- refused a path or a URL alone
- refused "it works", "tests pass"
- refused an anchor that does not resolve
- refused one box left `[ ]`

A probe that passes before the work warns (refused with `probes.base_must_fail`). The gate sees that evidence resolves, not that it proves the claim: that is the reviewer's job, and yours.

Two tracks

	client	prototype
Scope	`charter/scope.md` required, gate hard	none, gate off
Perimeter note	hard at commit	warns
Anchored reports	hard in CI	warns
Done, size, decisions	hard on both; rails optional on both	

Client only under a signed devis or contract. A prototype that gains a client: (O) writes `charter/scope.md` and sets `"track": "client"`.

Sources: `docs/gates.md` § Done needs evidence; `docs/method.md` § Two tracks.

When a gate refuses

You see	Who	Do this
`charter/done block` … `unchecked`	(A)	Run each probe, tick the box, cite the run under `## Evidence`; or keep the status open.
`charter/done block` … `names no commit … and no $ command`	(A)	Add the SHA and each probe's `$ command` line (`charter probe <id>` prints them). No flag overrides done; a unit closed without evidence is `parked` or `skipped`.
`charter/protected block [<path>]` at commit, or a denied Edit of an owner-only file	(A)→(O)	Leave it staged and hand over. In session, propose instead: a change request, a successor decision, a draft outside the project.
Out-of-scope path (client track)	(A)→(O)	Write `charter/changes/<slug>.md` from `charter/templates/change.md`, `status: proposed`; the owner approves.
Frozen file grew past its limit	(A)→(O)	Move code out; shrinking always passes. Or the owner opens a `kind: split` unit or a `size_exceptions` entry.
Retired decision cited, or a decision's `check:` fails	(A)→(O)	Cite the successor the message names; or write a new decision with `supersedes:`, and the owner retires the old one.
`charter/rails warn` … `names no rail` (refused with `rails.required`)	(A)→(O)	Set `rail: <open id>`. Work the road does not name waits: the owner adds a rail first.
`charter/config block`, or `problem:` from `charter config`	(O)	The owner corrects `charter/config.json`; the message names the key it meant (`did you mean `required`?`).
Stop hook: a done claim with no commit, `path:line` or run	(A)	Cite one, or say plainly what is not done. It blocks once per turn.
`charter check` prints one error line and passes	(O)	A gate crashed and failed open. `charter log`, and report it as a charter bug.

Sources: `docs/install.md` § When a gate refuses; `docs/gates.md` § The gates, § Config; messages from the 0.4.4 rehearsal.

In a project

Command	What it does	Writes
`charter next [--limit N]`	Where it stands, what comes next	nothing
`charter config`	Track, opt-ins; exit 1 on a problem	nothing
`charter version` · `charter unit show`	Installed version; the active unit	nothing
`charter log [--days N] [--verbose]`	What the gates did, 30 days	nothing
`charter synth`	The synthesis, printed	nothing
`charter synth --write`	Refused if a line is unanchored	`progress.md`
`charter unit start <id>` · `unit stop`	Attribute edits and cost	git dir
`charter probe <id> [--timeout S]`	Runs probes, prints `$` lines	gate log
`charter check --staged | --all [--base REF] [--strict]`	Every gate; `--all` runs each `check:`	gate log
`charter anchors <file> [--pr]`	Every claim has an anchor	gate log
`charter ledger fold` · `ledger add --human-minutes N --unit ID`	Ended sessions' cost; your time	`ledger.jsonl`
`charter decisions index` · `charter setup`	Decision index; git hook in this clone, once	`INDEX.md`; git config

(A) The agent never

sets `CHARTER_OWNER`, passes `--no-verify`, writes a refused file through Bash, runs `git add -A`, pushes, merges or deploys.

Sources: `docs/package.md` § Commands; `docs/install.md` § Rules for the agent; `docs/design/charter-next.md`; `cmd_*` in `package/.charter/bin/charter`.

From the charter checkout

```
charter-pkg survey    $P --track T --out DIR [--test-cmd C] [--keep-clone]
charter-pkg install   $P --track T [--dry-run]
charter-pkg upgrade   $P [--dry-run]
charter-pkg uninstall $P [--dry-run] [--purge-state]
charter-pkg status    $P
```

`--force` and `--replace-hooks`: only on the owner's word.

Upgrade

Step 0 again · `git switch -c charter-<version>` · read each upgrade note in `docs/package.md` · `upgrade --dry-run`, then `upgrade` · `charter config` prints no `problem:` · stage by name · (O) commits `charter <version>` · step 9. Rollback: `git revert`.

(O) Only the owner changes

`charter/scope.md`, `rails.md`, `config.json`, `.charter/**`, `.claude/settings*.json`, `.github/workflows/charter.yml`; a new id in `legacy-taskets.txt`; a change's approval fields; a decision's `status`, `superseded_by`, and an active one's `check:`, `binds:`; opening a `kind: split` unit. And every commitment: what is sold, price, dates, deploys, production data, releases, merging.

Skills in the project

`charter-unit` *"start unit" "resume" "finish the unit"*; `charter-decision` *"record the decision" "this is out of scope" "change request" "lesson learned"*; `charter-progress` *"where does this stand" "status for the client"*.

Sources: `docs/install.md` § Upgrade; `package/.claude/rules/charter.md:28-44`; `package/.claude/skills/*/SKILL.md`.

Personal layer, this machine only

`CharterAdmin`: *"charter status" "upgrade charter on X" "migrate X to charter" "fold the ledgers"*; open decisions → a Crossroads or Slate page.

`charter: package X available, Y installed`: X comes from whatever branch `~/Projects/charter` has checked out, so it can be unmerged. `waiting on you: …`: decide with `/slate` or `/crossroads`.

`/supervise --road [--infinite]`: works the road unattended, merging via `/reviewPR`. `--infinite` asks your questions first, hands over at the context cap, waits out usage limits, parks blocked units: only you stop it. Keys `on_block road_grant road_review`.

Sources: `personal/claude-home/Skills/CharterAdmin/SKILL.md`; `personal/claude-home/hooks/charter-personal.py`; `docs/daily-use.md` § Working the road.

Where it stops: evidence is checked for existence, not truth; the scope gate matches paths; a Bash edit skips the session hooks; gates fail open on their own errors (`docs/gates.md` § What a gate cannot see).

Checked against 0.4.4 and a rehearsal, 2026-09-25. `docs/cheatsheet.html`